



APPDEV LEC/LAB PRELIM EXAM A BUSINESS ESTABLISHMENT'S DATABASE

Smart Room Reservation & Access Control System for a Emilio Aguinaldo College - Cavite College

This system is used for managing the booking of shared campus rooms (classrooms, conference rooms, study halls, and labs) by students, faculty, and student organizations in EAC-C. When a user books a room, they gain temporary exclusive control over that room's devices (aircon, TV) for the duration of their slot. Users confirm their attendance through a QR code check-in; if no check-in happens within a grace period, staff can verify the room is empty and the booking is flagged as a no-show, freeing the room up for anyone on the waitlist. The system also tracks each user's trust score based on their booking history, so habitual no-shows lose booking priority over time.

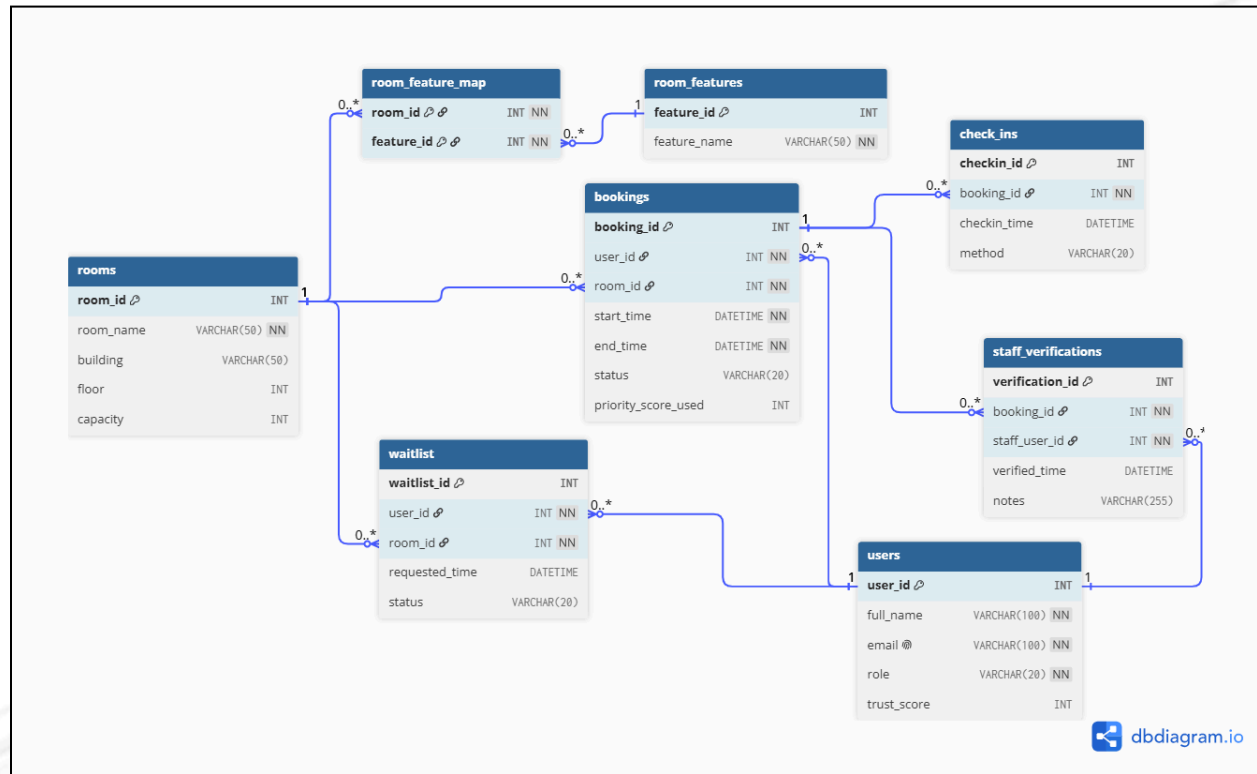
Cantiga, Ken Lester T.

UCOS 3 - 2



SCHOOL OF INFORMATICS AND COMPUTER STUDIES

DATA MODEL



This data model shows how rooms, users, and bookings connect to each other in the reservation system. A user can have many bookings and a room can be booked many times, but each room can have multiple features through the room_feature_map table which handles the many to many relationships. Bookings also connect to check_ins, staff_verifications, and waitlist since those all depend on a specific booking happening.



SCHOOL OF INFORMATICS AND COMPUTER STUDIES

SQL QUERIES

I. SQL QUERY FOR CREATING THE TABLES - [GITHUB LINK](#)

users

Programiz Online SQL Editor

Interactive SQL Course

Input

```
CREATE TABLE users (  
  user_id INTEGER PRIMARY KEY AUTOINCREMENT,  
  full_name VARCHAR(100) NOT NULL,  
  email VARCHAR(100) UNIQUE NOT NULL,  
  role VARCHAR(20) NOT NULL,  
  trust_score INT DEFAULT 100  
);
```

Run SQL

Available Tables

Users

user_id	full_name	email	role	trust_soc
empty				

Output

SQL query successfully executed. However, the result set is empty.

This table stores everyone who uses the system, like students, faculty, org reps, janitors, and admins. It also keeps a trust score that goes down when someone doesn't show up to their booking and goes up when someone willingly gives up a booked room, higher trust score gives higher priority on booking rooms.

rooms

Programiz Online SQL Editor

Interactive SQL Course

Input

```
CREATE TABLE rooms (  
  room_id INTEGER PRIMARY KEY AUTOINCREMENT,  
  room_code VARCHAR(10) NOT NULL,  
  building_number INT NOT NULL,  
  building_name VARCHAR(50),  
  floor INT NOT NULL,  
  room_number INT NOT NULL,  
  capacity INT  
);
```

Run SQL

Available Tables

Rooms

room_id	room_code	building_number	bu
empty			

Staff_verifications

verification_id	booking_id	staff_user_id
empty		

Users

user_id	full_name	email
1	Maria Santos	maria.santos@school.edu

Output

SQL query successfully executed. However, the result set is empty.

This table lists all the bookable rooms using the EAC-C room code format, like 3104 for building 3, floor 1, room 04. It also stores the building name and how many people the room can fit.



SCHOOL OF INFORMATICS AND COMPUTER STUDIES

room_features

Programiz Online SQL Editor Interactive SQL Course

> Input

```
CREATE TABLE room_features (  
    feature_id INTEGER PRIMARY KEY AUTOINCREMENT,  
    feature_name VARCHAR(50) NOT NULL  
);
```

Run SQL

Available Tables

Room_features

feature_id	feature_name
empty	

Rooms

room_id	room_name	building	floor	ca
empty				

Output

SQL query successfully executed. However, the result set is empty.

Users

user_id	full_name	email	role	trust_scc
empty				

This is a simple lookup table for the equipment a room can have, like aircon, TV, or projector.

room_feature_map

Programiz Online SQL Editor Interactive SQL Course

> Input

```
CREATE TABLE room_feature_map (  
    room_id INT NOT NULL,  
    feature_id INT NOT NULL,  
    PRIMARY KEY (room_id, feature_id),  
    FOREIGN KEY (room_id) REFERENCES rooms(room_id),  
    FOREIGN KEY (feature_id) REFERENCES room_features(feature_id)  
);
```

Run SQL

Available Tables

Room_feature_map

room_id	feature_id
empty	

Room_features

feature_id	feature_name
empty	

Rooms

room_id	room_name	building	floor	ca
empty				

Output

SQL query successfully executed. However, the result set is empty.

This connects rooms and features together since one room can have many features and one feature can belong to many rooms. It's the table that makes that many to many relationships work.



SCHOOL OF INFORMATICS AND COMPUTER STUDIES

bookings

Programiz Online SQL Editor Interactive SQL Course

Input

```
CREATE TABLE bookings (  
  booking_id INTEGER PRIMARY KEY AUTOINCREMENT,  
  user_id INT NOT NULL,  
  room_id INT NOT NULL,  
  start_time DATETIME NOT NULL,  
  end_time DATETIME NOT NULL,  
  status VARCHAR(20) DEFAULT 'confirmed',  
  priority_score_used INT DEFAULT 0,  
  FOREIGN KEY (user_id) REFERENCES users(user_id),  
);
```

Output

SQL query successfully executed. However, the result set is empty.

Available Tables

Bookings

booking_id	user_id	room_id	start_time
empty			

Room_feature_map

room_id	feature_id
empty	

Room_features

feature_id	feature_name
empty	

Rooms

This is the main table that records who booked which room and when. It also tracks the status of the booking, like confirmed, no-show, or cancelled.

check_ins

Programiz Online SQL Editor Interactive SQL Course

Input

```
CREATE TABLE check_ins (  
  checkin_id INTEGER PRIMARY KEY AUTOINCREMENT,  
  booking_id INT NOT NULL,  
  checkin_time DATETIME,  
  method VARCHAR(20),  
  FOREIGN KEY (booking_id) REFERENCES bookings(booking_id)  
);
```

Output

SQL query successfully executed. However, the result set is empty.

Available Tables

Bookings

booking_id	user_id	room_id	start_time
empty			

Check_ins

checkin_id	booking_id	checkin_time	me
empty			

Room_feature_map

room_id	feature_id
empty	

This logs when someone actually checks in to their booked room using a QR scan. If a booking has no check-in record, that usually means the person didn't show up.



SCHOOL OF INFORMATICS AND COMPUTER STUDIES

staff_verifications

Programiz Online SQL Editor Interactive SQL Course

Input

```
CREATE TABLE staff_verifications (  
  verification_id INTEGER PRIMARY KEY AUTOINCREMENT,  
  booking_id INT NOT NULL,  
  staff_user_id INT NOT NULL,  
  verified_time DATETIME,  
  notes VARCHAR(255),  
  FOREIGN KEY (booking_id) REFERENCES bookings(booking_id),  
  FOREIGN KEY (staff_user_id) REFERENCES users(user_id)  
);
```

Output

SQL query successfully executed. However, the result set is empty.

Available Tables

Rooms

room_id	room_name	building	floor
empty			

Staff_verifications

verification_id	booking_id	staff_user_id
empty		

Users

user_id	full_name	email	role	trust_s
empty				

This records when a staff member like a janitor or admin manually confirms whether a room was actually used or left empty. It's used to double check no-shows before marking them official.

waitlist

Programiz Online SQL Editor Interactive SQL Course

Input

```
CREATE TABLE waitlist (  
  waitlist_id INTEGER PRIMARY KEY AUTOINCREMENT,  
  user_id INT NOT NULL,  
  room_id INT NOT NULL,  
  requested_time DATETIME,  
  status VARCHAR(20) DEFAULT 'waiting',  
  FOREIGN KEY (user_id) REFERENCES users(user_id),  
  FOREIGN KEY (room_id) REFERENCES rooms(room_id)  
);
```

Output

SQL query successfully executed. However, the result set is empty.

Available Tables

Staff_verifications

verification_id	booking_id	staff_user_id
empty		

Users

user_id	full_name	email	role	trust_s
empty				

Waitlist

waitlist_id	user_id	room_id	requested_ti
empty			

This keeps track of users who want a room that's already taken, so it can be offered to them if the original booker doesn't show up.



SCHOOL OF INFORMATICS AND COMPUTER STUDIES

II. SQL QUERY FOR INSERTING RECORDS IN THE TABLES - [GITHUB LINK](#)

users

Programiz Online SQL Editor

Interactive SQL Course

Input

```
INSERT INTO users (full_name, email, role, trust_score) VALUES
('Maria Santos', 'maria.santos@school.edu', 'student', 95),
('John Dela Cruz', 'john.delacruz@school.edu', 'student', 60),
('Ana Reyes', 'ana.reyes@school.edu', 'faculty', 100),
('Carlo Mendoza', 'carlo.mendoza@school.edu', 'org', 80),
('Grace Villanueva', 'grace.villanueva@school.edu', 'janitor', 100),
('Rico Bautista', 'rico.bautista@school.edu', 'admin', 100);
```

Run SQL

Available Tables

Users

user_id	full_name	email
1	Maria Santos	maria.santos@school.edu
2	John Dela Cruz	john.delacruz@school.edu
3	Ana Reyes	ana.reyes@school.edu
4	Carlo Mendoza	carlo.mendoza@school.edu
5	Grace Villanueva	grace.villanueva@school.edu
6	Rico Bautista	rico.bautista@school.edu

Waitlist

Output

SQL query successfully executed. However, the result set is empty.

Added 6 sample users covering different roles like student, faculty, org, janitor, and admin, each with different trust scores.

rooms

Programiz Online SQL Editor

Interactive SQL Course

Input

```
INSERT INTO rooms (room_code, building_number, building_name, floor, room_number, capacity) VALUES
('3104', 3, 'Engineering', 1, 4, 25),
('2305', 2, 'Comp Lab', 3, 5, 25),
('5201', 5, 'Senior High', 2, 1, 30),
('6201', 6, 'Canteen / MPH', 2, 1, 100),
('7306', 7, 'Law', 3, 6, 30);
```

Run SQL

Available Tables

Rooms

room_id	room_code	building_number	bu
1	3104	3	En
2	2305	2	Cc
3	5201	5	Se
4	6201	6	Cc MI
5	7306	7	La

Staff_verifications

verification_id	booking_id	staff_user_id
empty		

Output

SQL query successfully executed. However, the result set is empty.

Added 5 rooms using real room codes based on our building layout, covering Engineering, Comp Lab, Senior High, Canteen, and Law.



SCHOOL OF INFORMATICS AND COMPUTER STUDIES

room_features

Programiz Online SQL Editor Interactive SQL Course

Input

```
INSERT INTO room_features (feature_name) VALUES  
('Aircon'),  
('TV'),  
('Projector');
```

Output

SQL query successfully executed. However, the result set is empty.

Available Tables

empty

Room_features

feature_id	feature_name
1	Aircon
2	TV
3	Projector

Rooms

room_id	room_code	building_number	bu
1	3104	3	En
2	2305	2	Cc
3	5201	5	Se
4	6201	6	Cc
			MI

Added 3 basic features that rooms can have: aircon, TV, and projector.

room_feature_map

Programiz Online SQL Editor Interactive SQL Course

Input

```
INSERT INTO room_feature_map (room_id, feature_id) VALUES  
(1, 1), (1, 2),  
(2, 1), (2, 3),  
(3, 1), (3, 2), (3, 3),  
(4, 3),  
(5, 1), (5, 3);
```

Output

SQL query successfully executed. However, the result set is empty.

Available Tables

empty

Room_feature_map

room_id	feature_id
1	1
1	2
2	1
2	3
3	1
3	2
3	3
4	3
5	1
5	3

Linked each room to the features it actually has, so some rooms have more than one feature.



SCHOOL OF INFORMATICS AND COMPUTER STUDIES

bookings

Programiz Online SQL Editor Interactive SQL Course

> Input

```
INSERT INTO bookings (user_id, room_id, start_time, end_time, status, priority_score_used) VALUES
(1, 1, '2026-09-14 09:00:00', '2026-09-14 10:00:00', 'completed', 5),
(2, 2, '2026-09-14 10:00:00', '2026-09-14 11:00:00', 'no-show', 5),
(3, 3, '2026-09-14 13:00:00', '2026-09-14 14:30:00', 'completed', 10),
(4, 1, '2026-09-15 09:00:00', '2026-09-15 10:00:00', 'cancelled', 5),
(1, 4, '2026-09-15 11:00:00', '2026-09-15 12:00:00', 'completed', 5),
(2, 5, '2026-09-15 14:00:00', '2026-09-15 15:00:00', 'no-show', 5);
```

Run SQL

Available Tables

Bookings

booking_id	user_id	room_id	start_time
1	1	1	2026-09-14 09:00:00
2	2	2	2026-09-14 10:00:00
3	3	3	2026-09-14 13:00:00
4	4	1	2026-09-15 09:00:00
5	1	4	2026-09-15 11:00:00
			2026-09-

Output

SQL query successfully executed. However, the result set is empty.

Added 6 sample bookings with different statuses, including some completed, some no-show, and one cancelled.

check_ins

Programiz Online SQL Editor Interactive SQL Course

> Input

```
INSERT INTO check_ins (booking_id, checkin_time, method) VALUES
(1, '2026-09-14 09:02:00', 'QR'),
(3, '2026-09-14 13:03:00', 'QR'),
(5, '2026-09-15 11:01:00', 'QR'),
(2, NULL, NULL),
(6, NULL, NULL);
```

Run SQL

Available Tables

Check_ins

checkin_id	booking_id	checkin_time	me
1	1	2026-09-14 09:02:00	QR
2	3	2026-09-14 13:03:00	QR
3	5	2026-09-15 11:01:00	QR
4	2		
5	6		

Room_feature_map

room_id	feature_id
---------	------------

Output

SQL query successfully executed. However, the result set is empty.

Added check-in records only for the bookings that were actually completed, since no-show bookings don't have a check-in.



SCHOOL OF INFORMATICS AND COMPUTER STUDIES

staff_verifications

Programiz Online SQL Editor Interactive SQL Course

Input

```
INSERT INTO staff_verifications (booking_id, staff_user_id, verified_time, notes) VALUES
(2, 5, '2026-09-14 10:15:00', 'Confirmed empty room, marked as no-show'),
(6, 5, '2026-09-15 14:15:00', 'Confirmed empty room, marked as no-show'),
(4, 6, '2026-09-15 09:10:00', 'Booking cancelled by user before start'),
(1, 5, '2026-09-14 09:05:00', 'Room in use as expected'),
(3, 6, '2026-09-14 13:05:00', 'Room in use as expected');
```

Run SQL

Available Tables

Staff_verifications

verification_id	booking_id	staff_user_id
1	2	5
2	6	5
3	4	6

Output

SQL query successfully executed. However, the result set is empty.

Added records showing staff manually verifying rooms after a booking, whether it was used properly or ended up empty.

waitlist

Programiz Online SQL Editor Interactive SQL Course

Input

```
INSERT INTO waitlist (user_id, room_id, requested_time, status) VALUES
(3, 2, '2026-09-14 10:05:00', 'claimed'),
(4, 2, '2026-09-14 10:05:00', 'expired'),
(2, 1, '2026-09-15 09:05:00', 'claimed'),
(1, 5, '2026-09-15 14:05:00', 'offered'),
(3, 5, '2026-09-15 14:05:00', 'waiting');
```

Run SQL

Available Tables

Waitlist

waitlist_id	user_id	room_id	requested_time
1	3	2	2026-09-14 10:05:00
2	4	2	2026-09-14 10:05:00
3	2	1	2026-09-15 09:05:00
4	1	5	2026-09-15 14:05:00
5	3	5	2026-09-15 14:05:00

Output

SQL query successfully executed. However, the result set is empty.

Added sample waitlist entries showing users waiting for a room, including ones that got claimed and one that expired.



SCHOOL OF INFORMATICS AND COMPUTER STUDIES

III. SQL QUERIES DEMONSTRATING WHERE, JOIN, GROUP BY SQL STATEMENTS.

Query 1 (JOIN)

Programiz Online SQL Editor

Interactive SQL Course

Input

```
SELECT
    b.booking_id, u.full_name AS requester,
    r.room_code, b.start_time,
    b.end_time, b.status
FROM bookings b
JOIN users u ON b.user_id = u.user_id
JOIN rooms r ON b.room_id = r.room_id
ORDER BY b.start_time;
```

Run SQL

Output

booking_id	requester	room_code	start_time	end_time	status
1	Maria Santos	3104	2026-09-14 09:00:00	2026-09-14 10:00:00	completed
2	John Dela Cruz	2305	2026-09-14 10:00:00	2026-09-14 11:00:00	no-show
3	Ana Reyes	5201	2026-09-14 13:00:00	2026-09-14 14:30:00	completed
4	Carlo Mendoza	3104	2026-09-15 09:00:00	2026-09-15 10:00:00	cancelled
5	Maria Santos	6201	2026-09-15 11:00:00	2026-09-15 12:00:00	completed
6	John Dela Cruz	7306	2026-09-15 14:00:00	2026-09-15 15:00:00	no-show

This shows all bookings together with the name of the person who booked and the room they booked, instead of just showing IDs.

Query 2 (WHERE)

Programiz Online SQL Editor

Interactive SQL Course

Input

```
SELECT booking_id, user_id, room_id, start_time, status
FROM bookings
WHERE status IN ('no-show', 'cancelled');
```

Run SQL

Output

booking_id	user_id	room_id	start_time	status
2	2	2	2026-09-14 10:00:00	no-show
4	4	1	2026-09-15 09:00:00	cancelled
6	2	5	2026-09-15 14:00:00	no-show

This filters out only the bookings that ended up as no-show or cancelled, which are the ones that matter for the waitlist logic.



SCHOOL OF INFORMATICS AND COMPUTER STUDIES

Query 3 (GROUP BY + COUNT)

Programiz Online SQL Editor Interactive SQL Course

Input

```
SELECT
    r.room_code,
    COUNT(b.booking_id) AS times_booked
FROM rooms r
LEFT JOIN bookings b ON r.room_id = b.room_id
GROUP BY r.room_code
ORDER BY times_booked DESC;
```

Output

room_code	times_booked
3104	2
7306	1
6201	1
5201	1
2305	1

This counts how many times each room has been booked, so we can see which rooms are the most requested.

Query 4 (GROUP BY + AVG)

Programiz Online SQL Editor Interactive SQL Course

Input

```
SELECT
    role,
    AVG(trust_score) AS avg_trust_score,
    COUNT(*) AS num_users
FROM users
GROUP BY role
ORDER BY avg_trust_score DESC;
```

Output

role	avg_trust_score	num_users
janitor	100	1
faculty	100	1
admin	100	1
org	80	1
student	77.5	2

This gets the average trust score per user role, to see which group tends to be more reliable with their bookings.



SCHOOL OF INFORMATICS AND COMPUTER STUDIES

Query 5 (JOIN + WHERE + GROUP BY)

Programiz Online SQL Editor

Interactive SQL Course

Input

```
SELECT
    r.room_code,
    COUNT(w.waitlist_id) AS waitlist_count
FROM waitlist w
JOIN rooms r ON w.room_id = r.room_id
WHERE w.status IN ('waiting', 'offered')
GROUP BY r.room_code
HAVING COUNT(w.waitlist_id) >= 1
ORDER BY waitlist_count DESC;
```

Output

room_code	waitlist_count
7306	2

This counts how many people are on the waitlist for each room, but only counts the ones still waiting or offered a slot.